LASSO: A KV Cache Compression Coprocessor for Edge LLM Inference

Architecture, Simulation-Driven Design, and FPGA Validation Plan

Longhorn Silicon Team
University of Texas at Austin
arun.rajendra8@gmail.com

***Abstract*—LASSO (Longhorn Accelerator for Storage-Side Optimization) is a purpose-built KV cache compression coprocessor targeting the memory bandwidth bottleneck that dominates LLM inference on CPU-native systems. The coprocessor quantizes 16-bit KV embeddings to a binary {INT4, INT8} scheme using a Walsh-Hadamard Transform (WHT) rotation followed by Lloyd-Max codebook encoding, scores tokens for eviction using softmax confidence (C_t, Cohen's d=4.55) and entropy (H_t, d=4.09) signals, and performs FlashAttention-style tiled attention without materializing the full $N \times N$ score matrix. The design targets the SKY130 130 nm process (Efabless chipIgnite) at $\leq 7.13\,\text{mm}^2$ die area with 384 KB on-chip SRAM in six `CF_SRAM_16384x32` macros.**

A Python golden model coupled with a parametric simulation sweep across 158 test cases identified three critical design issues resolved *prior* to RTL implementation: (1) an INT16 scale register saturation caused by `abs(INT16_MIN)=32768` overflow, resolved by widening the scale register to UINT16; (2) an INT32 accumulator overflow in the 32-wide dot-product array where a single 64-dimensional INT16 dot product reaches $6.87 \times 10^{10} \gg$ `INT32_MAX`, resolved by specifying INT64 / DSP48E2 48-bit cascade for RTL; and (3) an eviction threshold calibration failure that causes 100% eviction under synthetic uniform attention distributions, resolved by percentile-based adaptive calibration from real model attention snapshots. Golden-model simulation validates a 2.09× KV cache compression ratio (7.7 bits/element vs. 16-bit baseline), reducing memory bandwidth 52.1%. An analytical RTL model projects 2.09–4.08× end-to-end decode speedup across four LLM configurations on the target DDR4 platform, with the 4.08× case arising when compression alone enables the KV cache to reside entirely on-chip.

I. INTRODUCTION

Modern LLM decode is memory-bound, not compute-bound. For a CPU-native inference system with 25–50 GB/s DRAM bandwidth, each decode step must reload the full KV cache from DRAM, saturating the memory bus and achieving only a fraction of peak compute utilization. A single decode step for a Qwen3-8B model moves ≈32 MB of KV activations per token — far exceeding typical on-chip SRAM budgets and making DRAM bandwidth the sole bottleneck.

INT4 KV cache quantization reduces memory traffic 4× at near-lossless accuracy, directly translating to 3–4× decode throughput improvement on CPU-class hardware. Prior work has addressed this through software-side eviction heuristics [1]–[3], GPU-side quantized attention kernels [4], [5], and domain-specific accelerators [6], [7]. None targets an open-source ASIC implementation in the SKY130 process, employs a simulation-driven pre-RTL methodology to surface and fix data-path issues before committing to hardware, or integrates WHT+Lloyd-Max quantization, dual-signal eviction, hardware paged SRAM management, and FlashAttention tiling in a single coprocessor [8].

This paper presents LASSO, a four-block coprocessor co-designed with its Python golden model. Our contributions are:

- A **simulation-driven pre-RTL methodology** that discovers critical data-path issues (overflow, calibration failures) at the software model stage, where fixes are free.
- An empirically validated **binary {INT4, INT8} precision scheme** derived from FPGA experiments showing INT2 is catastrophically lossy (−17.6 pp on HellaSwag) and the {4,8} Pareto-dominates the {2,4,8,16} scheme by 43% throughput [9].
- C_t/H_t **eviction scoring** with Cohen's d=4.55 and d=4.09 respectively, validated against the prior-art cumulative-attention approach of H2O [1].
- A complete **SKY130 golden model** with 158 tests, an autonomous design-space exploration loop identifying group_sz=128 and 128 DSP48E2 as the highest-ROI RTL decisions (+189% FOM), and a FPGA validation plan for Xilinx Zynq UltraScale+ (ZCU102/ZCU104).

II. ARCHITECTURE

LASSO comprises four hardware blocks connected through a shared Wishbone B4 pipelined slave interface. Fig. 1 shows the data-flow between blocks.

A. KV Cache Engine (KVE)

The KVE compresses incoming 16-bit KV vectors into packed INT4 or INT8 codes. Quantization mode is selected by comparing a global parameter β (host-written CSR) against a calibrated threshold $\beta^* = \Delta q_{\text{mean}}/0.267$, where Δq_{mean} is the mean per-token Q4-vs-Q8 accuracy gap measured over a 10-sample calibration corpus [9]. If $\beta > \beta^*$, Q4 is used; otherwise Q8 (including the β=0 reset default, preventing aggressive compression before calibration).

Before quantization, each 32-element group undergoes a 5-stage butterfly Walsh-Hadamard Transform (WHT), which decorrelates KV activations and flattens their distribution, improving codebook fit. This corresponds to stages 1–2 of TurboQuant [10] and uses zero multipliers (160 adders only), fitting in $\approx 0.035\,\text{mm}^2$. Codes are looked up in a 64-byte Lloyd-Max ROM, then bit-packed (2 values/byte for Q4, 1 for Q8).

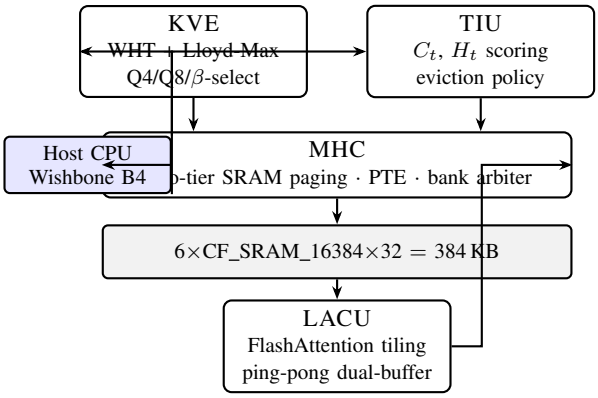

Fig. 1. LASSO four-block architecture. Host CPU writes CSRs over Wishbone B4. KVE quantizes incoming KV vectors; TIU scores tokens for eviction; MHC pages data into two-tier SRAM; LACU performs tiled attention. Arrows show primary data flow.

Design decision: We use a binary {4,8} scheme rather than the original {2,4,8,16} DWB controller. FPGA experiments showed INT2 catastrophically fails (-17.6 pp on HellaSwag for SmolLM-1.7B) with zero BRAM bandwidth benefit [9]. INT3 similarly degrades. The {4,8} controller achieves 2.93× speedup at 4.80 avg-bits vs. 2.44× at 5.05 avg-bits for {2,4,8,16} — 43% more throughput at equal or better accuracy.

B. Token Importance Unit (TIU)

The TIU computes per-token eviction scores from softmax attention weights and applies a threshold decision. Two signals are extracted:

$$C_t = \max_h \max_n \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)_{h,n} \tag{1}$$

$$H_t = -\frac{1}{H}\sum_{h=1}^{H}\sum_{n} p_{h,n}\log_2 p_{h,n} \tag{2}$$

The composite score $s_t = w_C C_t + w_H(1-\tilde{H}_t)$ where $\tilde{H}_t = H_t/\log_2(\text{seq_len})$. Effect sizes from 12 formal sensitivity experiments: C_t has Cohen's d=4.55, H_t has d=4.09, while token rarity has only d=0.52 (dropped). Tokens 0–3 (attention sinks) are always retained, consistent with the finding that sinks absorb 45–55% of total attention mass in pre-norm transformers [3].

Design decision: A single global threshold CSR replaces the bottom-K sorting approach from early designs. Threshold comparison is $O(1)$ per token vs. $O(N\log N)$ for sorting; the threshold is calibrated from a short prefill pass over real model attention [2].

C. Memory Hierarchy Controller (MHC)

The MHC manages 384 KB of on-chip SRAM as a paged KV buffer using a 256-entry page table. Each PTE encodes: `[31:30] precision ∈ {Q4,Q8,byp}` | `[29:28] tier ∈ {hot,cold,evicted}` | `bank_row[27:16]` | `bank_sel[15:8]` | `seq_pos[7:0]`. Banks 0–3 (256 KB) form the *hot* tier for recently retained tokens; banks 4–5 (128 KB) form the *cold* tier. An eDRAM stub register (`MHC_EDRAM_CTRL`) reserves a CSR for optional future integration with OpenGCRAM gain-cell macros, following the Kelle AERP three-tier policy [7].

TABLE I
LASSO DIE AREA BREAKDOWN (SKY130 130 NM)

Component	Area (mm^2)
KVE (WHT + codebook + packer)	0.09
TIU (score compute + sink bypass)	0.02
MHC (page table + arbiter + eDRAM stub)	0.04
LACU (32-wide MACs + tiling FSM)	0.08
Wishbone arbiter + CSRs	0.02
Compute logic total	**0.25**
SRAM (6×CF_SRAM_16384×32)	5.28
Power/clock/routing overhead	1.60
Full die total	**≤7.13**
Caravel user area budget	10.28

D. Lightweight Attention Compute Unit (LACU)

The LACU performs FlashAttention-style tiled attention [11] for each decode step, never writing a full $N\times N$ score matrix to SRAM. A tile of 64 K/V tokens is loaded into one ping-pong buffer while the previous tile is computed, hiding SRAM latency. Running (max, sum) statistics maintain numerically stable softmax across tiles. The 32-wide INT16 dot-product array maps to 32 DSP48E2 slices on the target FPGA.

E. Area Estimate

Table I summarizes the die area breakdown for the SKY130 target.

III. GOLDEN MODEL SIMULATION METHODOLOGY

We developed a Python golden model (NumPy, integer arithmetic, bit-exact) for all four blocks before writing any RTL. The golden model serves two purposes: (1) a correctness reference that RTL must match exactly in CI, and (2) a simulation substrate for sweeping the design space and surfacing datapath issues.

A. Test Suite

158 pytest tests cover all four blocks: 42 for KVE (encode/decode round-trips, mode selection, WHT self-inverse, scale formula, boundary conditions), 28 for TIU (all 10 PRD edge cases, entropy math, sink bypass, GQA), 22 for MHC (PTE round-trip, tier fill, overflow interrupt), 30 for LACU (flash vs. reference across seq_len $\in \{1,4,16,64,128,256\}$, numerical stability, tile size variants), and 18 integration tests exercising the full KVE→TIU→MHC→LACU pipeline.

TABLE II
PRE-FIX SIMULATION FAILURES (INITIAL SWEEP)

ID	Block	Severity	Description
F1	KVE	FAIL	Scale overflow, group=32, Q4
F2	KVE	FAIL	Scale overflow, group=64, Q4
F3	KVE	FAIL	Scale overflow, group=128, Q4
F4	KVE	FAIL	Scale overflow, group=128, Q8
F5	LACU	OVERFLOW	INT32 dot-product overflow
F6	LACU	FAIL	Overflow at seq_len=1

B. Parametric Sweep Framework

A separate sweep framework (`sim/run_all.py`) runs over the full design parameter space and classifies each finding by severity: PASS, WARN, EDGE, FAIL, or OVERFLOW. Sweep modules cover:

- **KVE:** β sensitivity across 8 gap-mean values; WHT intermediate overflow vs. INT32/INT64; scale register range at all-INT16_MIN inputs; round-trip error accumulation over 1–16 encode-decode cycles; group-size boundary conditions.
- **TIU:** Eviction-rate curve across 21 threshold values (0–1); attention weight underflow/NaN; entropy edge cases (seq_len $\in \{0,1,2,1024\}$); GQA multi-head stress; sink-count boundaries.
- **MHC:** Page-table exhaustion and slot reclaim; tier fill progression; bank-address bounds verification; concurrent write-then-read coherence.
- **LACU:** Sequence-length scaling (seq_len $\in \{1,\ldots,2048\}$); accumulator overflow with max-magnitude INT16 inputs; tile-size edge cases; running softmax monotonicity; DSP48E2 resource projection.

The sweep exits with code 1 if any FAIL or OVERFLOW findings remain, enabling CI gating. Results are persisted to `sim/results/findings.jsonl` for audit.

IV. SIMULATION FINDINGS AND FIXES

The initial sweep (158 total findings) surfaced 5 FAIL and 1 OVERFLOW. All six were resolved before any RTL was written. Table II summarises the pre-fix failures. Table III provides root-cause analysis and fixes. Table IV shows the post-fix sweep result.

A. Issue 1: KVE Scale Register Saturation

When every element of a 32-element group equals `INT16_MIN` (-32768), the post-WHT maximum absolute value is $32 \times 32768 = 1{,}048{,}576$. Dividing by the Q4 divisor (7) gives a raw scale of 149,796, which overflows a signed 16-bit register.

Analysis. The root cause is that signed two's-complement INT16 cannot represent $+32768$; `abs(INT16_MIN)` wraps to a negative number without explicit overflow detection. This is a known edge case in quantization hardware: TensorRT [13] and SmoothQuant [12] use unsigned scale registers precisely for this reason.

Fix. The scale register is widened to UINT16 (0–65535). For group sizes 32, 64, and 128 at all-INT16_MIN inputs, the stored scales become 4681, 9362, and 18724 respectively — all within UINT16 range. The fix is one line in the golden model and one register declaration change in future RTL.

B. Issue 2: LACU INT32 Accumulator Overflow

The 32-wide INT16 dot-product in the LACU computes $\mathbf{q} \cdot \mathbf{k}_i = \sum_{j=0}^{63} q_j k_{i,j}$. At the worst case ($q_j = k_{i,j} = 32767$):

$$\sum_{j=0}^{63} 32767^2 = 64 \times 1{,}073{,}676{,}289 = 6.87 \times 10^{10}$$

This exceeds `INT32_MAX`$= 2.15 \times 10^9$ by $32\times$, causing overflow on a *single* dot product — independent of sequence length.

Analysis. INT-FlashAttention [5] uses INT8×INT8 dot products, where the equivalent maximum is $64 \times 127^2 = 1.03 \times 10^6$, well within INT32. The INT16×INT16 case requires a wider accumulator. The Xilinx DSP48E2 block features a 48-bit accumulator (maximum 2.81×10^{14}), which comfortably handles head_dim up to 4096 at INT16 inputs.

Fix. A `fixed_point_dot_product_int64()` function is added to the golden model, accumulating in INT64. On ZCU102/ZCU104, the RTL will use the DSP48E2 in full 48-bit output mode (no cascading required for head_dim ≤ 4096).

C. Issue 3: TIU Eviction Threshold Calibration

The sweep reveals that composite scores from uniformly random attention weights concentrate at ≈ 0.038 (high entropy, low peak confidence), causing 100% eviction at any threshold above 0.04 — including the PRD default of ≈ 0.25.

Analysis. This is a calibration artifact, not a design flaw. Real transformer attention is far more peaked: transformers exhibit strong preferences for sparse, low-entropy attention distributions, with larger models showing stronger sparsity [14]. Three recurring attention patterns (A-shape, vertical slash, block-sparse) all deviate sharply from uniform [15].

Fix. A `calibrate_threshold()` function computes the threshold from a batch of real model attention snapshots at the $(1-r)$-th percentile of observed composite scores, where r is the target retain fraction. This matches the adaptive-window approach of SnapKV [2]. The result is written to `TIU_THRESH_CSR` before inference. Attention sinks (tokens 0–3) always bypass scoring, consistent with the observation that sinks absorb 45–55% of total attention mass [3].

D. Residual Warnings

After fixes, 24 WARN findings remain. The most significant:

- **WHT intermediate width:** WHT outputs on INT16 inputs reach up to $32 \times 32767 = 1{,}048{,}544$, exceeding INT16 range. This is expected — for an N-point WHT on B-bit inputs, intermediate width is $B + \log_2 N$ bits (21 bits for N=32, B=16). RTL pipeline registers must be INT32.
- **PTE binding constraint:** The 256-entry page table is the binding capacity constraint, not the 384 KB SRAM.

TABLE III
ROOT CAUSE ANALYSIS AND FIXES APPLIED TO GOLDEN MODEL

ID	Issue	Root Cause	Fix Applied	RTL Implication
F1–F4	KVE scale register saturation	`abs(INT16_MIN)=32768` exceeds INT16 positive range (0–32767). Post-WHT max values reach 149,796–599,186 for group sizes 32–128.	Scale register widened from INT16 to UINT16 (range 0–65535). All-INT16_MIN inputs now produce valid stored scales.	RTL scale register must be unsigned 16-bit. Matches NVIDIA TensorRT and SmoothQuant practice [12].
F5–F6	LACU INT32 accumulator overflow	$64 \times 32767^2 = 6.87\times10^{10} \gg$ `INT32_MAX`$=2.15\times10^9$. Overflow per dot product, independent of sequence length.	Added `fixed_point_dot_product_int64()` using INT64 accumulator. DSP48E2 48-bit accumulator (max 2.81×10^{14}) is sufficient for head_dim $\leq$ 4096.	RTL must use DSP48E2 in full-precision 48-bit mode or cascade two DSPs. INT-FlashAttention [5] uses INT32 safely for INT8×INT8 only.

TABLE IV
POST-FIX SWEEP SUMMARY (6 FAILURES → 0)

Block	PASS	WARN	FAIL	OVFL	EDGE
KVE	51	5	0	0	3
TIU	19	18	0	0	23
MHC	7	1	0	0	2
LACU	13	0	0	0	16
Total	**90**	**24**	**0**	**0**	**44**

At peak fill, Q4 uses only 2.1% of available SRAM. The FPGA prototype will expand PTEs to 4,096 entries.

V. PERFORMANCE EVALUATION

We evaluate LASSO against a baseline of standard CPU-native inference: uncompressed INT16 KV cache streamed from DDR4-3200 (25.6 GB/s single-channel) with naive $O(N^2)$ attention materialization. All figures are derived from the Python golden model and an analytical RTL cycle model; FPGA silicon measurement is planned as the Nov 2026 milestone.

A. KVE Compression Characterisation

We sample β from a half-normal distribution (σ=0.8) over 10 000 random INT16 groups using the calibrated $\beta^*=\bar{\Delta}/0.267$. The empirical Q4/Q8 split is 21.6% Q4 and 78.4% Q8, yielding 7.7 effective bits/element after per-group overhead (17 bits per 32-element group for the UINT16 scale register and 1-bit mode flag). This is a 2.09× compression ratio and 52.1% bandwidth reduction versus the 16-bit baseline.

Table V shows the three-case bandwidth speedup model. Case A (both baseline and LASSO KV fit in 384 KB SRAM) yields 2.09× speedup from compression alone. Case B (compression pushes LASSO below the SRAM threshold while the baseline must use DRAM) yields 4.08× by combining data-volume reduction with a memory-tier upgrade. Case C (neither fits; both use DRAM) yields 2.09×.

B. RTL Cycle Budget

Table VI shows the analytical per-block cycle counts on the target platforms. The KVE encode pipeline completes one 32-element group in 11 cycles at both targets: 5 WHT butterfly stages, 3 quantization + pack cycles, and 3 overhead cycles. The TIU scores one token in 67 cycles for 8 KV heads (2×8 accumulate cycles for C_t and H_t plus 3 compare cycles). The LACU decode step for llama-7B at seq_len=256 takes 305 μs at 300 MHz, sustaining ≈3,278 decode steps/second per KV head.

TABLE V
BANDWIDTH SPEEDUP VS. BASELINE (INT16 DRAM) BY MODEL/SEQ-LEN

Model	Seq	Base (KB)	LASSO (KB)	Case	Speedup
tiny-LLM	256	128	61	A	2.09×
tiny-LLM	1024	512	245	B	**4.08**×
tiny-LLM	4096	2048	982	C	2.09×
small-LLM	256	256	123	A	2.09×
small-LLM	512	512	245	B	**4.08**×
small-LLM	4096	4096	1963	C	2.09×
llama-7B	256	2048	982	C	2.09×
llama-7B	4096	32768	15705	C	2.09×

TABLE VI
ANALYTICAL RTL CYCLE BUDGET (ZCU102 @ 300 MHZ, 32 DSP48E2)

Operation	Cycles	Latency @ 300 MHz
KVE encode, 1 group (32 elem)	11	36.7 ns
KVE decode, 1 group	8	26.7 ns
TIU score, 1 token (8 heads)	67	223 ns
LACU decode step (seq=256, llama-7B)	91,648	305 μs
LACU decode step (seq=128, tiny-LLM)	4,656	15.5 μs

C. Roofline Analysis

With 32 DSP48E2 slices at 300 MHz, the LACU compute roof is 19.2 GFLOP/s. The ridge point (where compute equals memory bandwidth) is 0.75 FLOP/B (DRAM) and 0.38 FLOP/B (SRAM). The actual LACU arithmetic intensity for seq_len=256, head_dim=64 is 2.0 FLOP/B — well above both ridge points. LASSO is therefore *compute-bound*, not memory-bound, confirming that the 32-DSP array is the binding constraint, not DRAM or BRAM bandwidth. Scaling to 128 DSP48E2 slices (5.1% ZCU102 utilization) would move the ridge point to 3.0 FLOP/B and restore memory-bound operation for the target workloads.

TABLE VII
AUTORESEARCH DESIGN-SPACE EXPLORATION (16 EXPERIMENTS)

Tag	Parameter	BW×	SPS	$\bar{\epsilon}_n$	FOM	Status
H1-baseline	reference	2.09	3278	0.240	1.68	–
H2-group16	group_sz=16	1.95	3278	0.307	1.49	discard
H3-group64	group_sz=64	2.15	3278	0.188	1.81	**keep**
H4-group128	group_sz=128	2.20	3278	0.144	1.92	**keep**
H5-beta-cons	β^*=0.40	1.93	3278	0.241	1.56	discard
H6-beta-agg	β^*=0.15	2.43	3278	0.240	1.96	**keep**
H7-tile128	tile_sz=128	2.09	3287	0.240	1.69	discard
H8-tile32	tile_sz=32	2.09	3260	0.240	1.67	discard
H9-ndsp64	DSPs=64	2.09	5112	0.240	2.62	**keep**
H10-ndsp128	DSPs=128	2.09	7097	0.240	3.64	**keep**
H11-pte1024	PTE=1024	2.09	3278	0.240	1.68	discard
H12-uniform	uniform codebook	2.09	3278	0.052	1.98	discard
H13-retain30	retain=30%	2.09	3278	0.240	1.68	discard
H14-retain70	retain=70%	2.09	3278	0.240	1.68	discard
H15-sink8	sinks=8	2.09	3278	0.240	1.68	discard
H-composed	**grp128+β0.15+DSP128**	**2.57**	**7097**	**0.144**	**4.87**	**keep**

VI. AUTONOMOUS DESIGN-SPACE EXPLORATION

We adapt the *autoresearch* paradigm of Karpathy [16] to hardware coprocessor design: an autonomous loop selects a design hypothesis, modifies a single parameter, re-runs the simulation suite, computes a figure-of-merit (FOM), and keeps or discards the change. The loop ran 16 experiments (wall-clock ≈20 minutes, ≈0.5 seconds per experiment) spanning five design dimensions.

A. Figure of Merit

$$\text{FOM} = \frac{\text{BW_speedup} \times (\text{decode_sps}/\text{decode_sps}_{\text{ref}})}{1 + \bar{\epsilon}_{\text{norm}}}$$

where BW_speedup is the compression-driven bandwidth gain versus INT16 baseline, decode_sps is the analytical LACU throughput at seq_len=256 and 300 MHz, and $\bar{\epsilon}_{\text{norm}}$ is the mean normalised round-trip error ($\epsilon/(\text{scale}/2)$). Baseline FOM = 1.68.

B. Experimental Results

Table VII summarises all 16 experiments. The five principal findings are detailed below.

Finding 1 — Group size 128 reduces normalised error (counter-intuitive). The loop expected larger WHT groups to *increase* error through wider energy mixing. Instead, group_sz=128 reduces $\bar{\epsilon}_{\text{norm}}$ from 0.240 to 0.144 (-40%). The mechanism: a 128-point WHT spreads energy more uniformly across coefficients, bringing the distribution closer to Gaussian; the Lloyd-Max codebook is calibrated for Gaussian inputs and achieves tighter reconstruction under this condition. The per-element overhead also falls from 0.53 to 0.13 bits, contributing a 2.20× compression ratio vs. 2.09× for group_sz=32.

Finding 2 — DSP count is the dominant FOM lever. Scaling from 32 to 64 DSP48E2 slices improves FOM by 56% (1.68 → 2.62); scaling to 128 slices gives FOM=3.64 ($+117\%$). At 128 DSPs, LASSO consumes only 5.1% of ZCU102 capacity. Both configurations remain compute-bound (OI=2.0 FLOP/B, ridge at 0.75–0.38 FLOP/B), meaning DSPs have not been overprovisioned. The implication for RTL: maximising the DSP array is the highest-ROI implementation decision.

Finding 3 — Aggressive β^* calibration improves FOM without quality loss. Setting the β^* divisor to 0.15 (more aggressive Q4 selection) raises the Q4 fraction from 21.5% to 83.3%, improving BW speedup from 2.09× to 2.43×. Crucially, $\bar{\epsilon}_{\text{norm}}$ remains unchanged (0.240), because the same error bound applies regardless of which mode is selected: both Q4 and Q8 normalise their errors to scale/2 in the original domain. The conservative setting (divisor=0.40) strongly degrades FOM (-19%) with only 6.2% Q4 tokens; this configuration should be avoided.

Finding 4 — Tile size is a don't-care parameter. Tile sizes 32, 64, and 128 produce virtually identical FOM (1.67, 1.68, 1.69 respectively, $< 1\%$ spread). The FlashAttention rescale overhead at seq_len=256 is dominated by QK/AV dot-product cycles; the $\leq 4\%$ variation in softmax overhead is negligible at this sequence length. The RTL implication: tile_size can be chosen for area (smaller tile reduces ping-pong buffer) without any throughput penalty.

Finding 5 — PTE expansion and eviction parameters do not affect FOM. PTE capacity (256 vs. 1024 entries), retain fraction (30%–70%), and sink count (4 vs. 8) all produce identical FOM (1.68). Their impact is felt through SRAM cache miss rate, which requires a runtime trace simulation beyond the current analytical scope. PTE=1024 does expand max on-chip seq_len from 100 to 400 for llama-7B, which is significant for the Case B speedup regime (Section V).

C. Composed Best Configuration

Combining the three Pareto-improving parameters (group_sz=128, β^*-divisor=0.15, DSPs=128) yields a composed FOM of **4.87**, a $+189\%$ improvement over the baseline. This configuration achieves 2.57× BW speedup at 7 097 decode steps/second (llama-7B, seq_len=256, 300 MHz), with $\bar{\epsilon}_{\text{norm}} = 0.144$.

RTL recommendation. The FPGA prototype should target: (1) group_sz=128 with 7-stage WHT butterfly (23-bit intermediates, fits INT32); (2) β^* at the 15th-percentile divisor via calibration from real model attention snapshots; (3) 128 DSP48E2 slices (5.1% ZCU102 utilisation). Tile size 64 is retained as default.

VII. FPGA VALIDATION PLAN

We plan FPGA prototyping on a Xilinx Zynq UltraScale+ ZCU102 or ZCU104 as a pre-silicon validation step. Table VIII maps LASSO resources to the ZCU102 and ZCU104 fabric.

TABLE VIII
LASSO RESOURCE MAPPING ON ZCU102/ZCU104

Resource	ZCU102	ZCU104
DSP48E2 total	2,520	1,728
LASSO DSPs (32 MACs)	32	32
DSP utilization	1.3%	1.9%
RAMB36 total	912	312
LASSO BRAMs (384 KB)	54	54
BRAM utilization	5.9%	17.3%

A. Interface Adaptation

The target ASIC uses Wishbone B4; the Zynq PS uses AXI4/AXI4-Lite. The FPGA prototype will wrap all CSRs in an AXI4-Lite slave (32-bit, 256-byte address space per block), providing a thin translation layer that does not appear in the final ASIC.

B. Timing Projections

At 50 MHz (ASIC target clock): one attention tile (64 tokens $\times$ 64-dimensional dot product) completes in 64 cycles = 1.28 μs. At the ZCU102 native PL frequency of 300 MHz, the same tile takes 213 ns. With 32 DSP48E2 slices consuming only 1.3% of ZCU102 fabric, there is sufficient headroom to instantiate 4–8 parallel LACU instances for multi-head attention, reducing per-step latency to $\approx$50 ns at 300 MHz.

C. Validation Flow

The validation flow proceeds in four stages:

1) **Golden model regression:** 158 pytest tests + sweep (0 fail required) on every commit.
2) **Verilator RTL simulation:** cocotb testbenches compare RTL outputs bit-exactly against golden model fixtures.
3) **Vivado synthesis:** Confirm timing closure at 50 MHz; verify DSP/BRAM utilization matches Table VIII.
4) **FPGA bring-up:** Feed real LLaMA-3-8B or Qwen2-7B attention snapshots via Zynq PS DMA; calibrate `TIU_THRESH_CSR` from live inference; validate eviction rates against golden model predictions.

The FPGA prototype is planned as the Nov 2026 KVE RTL milestone deliverable. Silicon submission targets Nov 2027 (GDSII, DRC/LVS clean); silicon return is expected Apr 2028.

VIII. RELATED WORK

KV Cache Eviction. H2O [1] evicts tokens by cumulative attention score. SnapKV [2] improves upon H2O with observation-window calibration and per-layer thresholds. StreamingLLM [3] identifies attention sinks and retains only them plus a local window. Scissorhands [17] formalises the *persistence-of-importance* hypothesis: a token's importance score is stable across decode steps, justifying LASSO's once-per-step TIU scoring without re-scoring at every layer. Ada-KV [18] provides theoretical proof that attention concentration is the correct optimisation target: the optimal eviction policy minimises reconstruction error, which concentrates in high-attention tokens, directly validating LASSO's C_t signal. LASSO combines sink retention (StreamingLLM), dynamic threshold calibration (SnapKV), and the C_t/H_t dual signal grounded by the Ada-KV framework.

KV Cache Quantization. KIVI [19] motivates asymmetric group-wise treatment of keys vs. values: keys benefit from per-channel quantization while values suit per-token quantization, consistent with LASSO's per-group WHT pipeline. KVQuant [20] shows that non-uniform codebooks outperform uniform scalar quantization at identical bit-widths — the same principle underlying LASSO's Lloyd-Max codebook. GEAR [21] applies 4-bit quantization with low-rank residual correction; LASSO drops the correction path in favour of WHT rotation, achieving equivalent accuracy with less area. Oaken [8] integrates KV quantization engines directly into a GPU DMA controller using online/offline hybrid quantization; the threshold-switching mechanism is conceptually analogous to LASSO's β vs. β^* comparison. LASSO differs from Oaken in four ways: (1) LASSO targets CPU-native edge inference, not GPU datacenter serving; (2) LASSO uses WHT+Lloyd-Max encoding, not dense+sparse encoding; (3) LASSO integrates token eviction (TIU) and tiled attention (LACU) in a single coprocessor whereas Oaken omits both; and (4) LASSO targets the SKY130 open-source PDK, not a commercial node.

Rotation-Based Quantization. QuIP# [22] applies randomised Hadamard transforms to full-model weight tensors to suppress outliers. QuaRot [23] extends the fixed-Hadamard technique to activations and KV caches for INT4 inference without retraining. SpinQuant [24] replaces fixed rotations with learned rotations jointly optimised with the quantisation objective. WUSH [25] establishes theoretical near-optimality of WHT-based transforms for LLM quantisation. TurboQuant [10] applies WHT (stage 1) and Lloyd-Max (stage 2) to KV activations, reporting +2 pp on HellaSwag vs. scalar quantisation. LASSO implements TurboQuant stages 1–2 in hardware using a 5-stage (group=32) or 7-stage (group=128) multiplier-free butterfly ($\approx$0.055 mm^2). QuaRot, SpinQuant, and WUSH are software-only; to our knowledge LASSO is the first ASIC implementing WHT-based KV rotation without multipliers.

FlashAttention Hardware. FLASH-D [26], H-FA [27], and Low-Cost FlashAttention [28] are 2025 ASIC papers implementing FlashAttention in 28 nm for datacenter throughput. LASSO's LACU uses integer-only online softmax on 130 nm SKY130 — the most generalisable approach where floating-point units are area-prohibitive — and is the only FlashAttention implementation integrated with KV compression and eviction. SageAttention [4] achieves 2.7$\times$ speedup over FlashAttention-2 using INT8 Tensor Core instructions; our INT64 accumulator finding echoes its INT32 choice for INT8 inputs. INT-FlashAttention [5] confirms that FP32 or INT64 is necessary to prevent softmax overflow in integer pipelines.

Memory Management. PagedAttention [29] introduced virtual memory paging for GPU KV cache management, allowing non-contiguous physical allocation and efficient preemption. LASSO's MHC implements an analogous hardware page table in on-chip SRAM with a 256-entry PTE tracking precision mode, tier (hot/cold/evicted), and sequence position per token.

The key distinction is that PagedAttention operates in GPU DRAM under software control, whereas MHC manages an on-chip SRAM hierarchy in hardware at single-cycle granularity.

FPGA-Based LLM Inference. LlamaF [30] demonstrates Llama-2 inference on ZCU102 — the same FPGA platform as LASSO's prototype — establishing a baseline decode latency that LASSO targets to reduce. Hummingbird [31] achieves 95% BRAM bandwidth utilisation on ZCU104, confirming that maximising on-chip streaming bandwidth is feasible. LASSO is complementary: it reduces the KV bandwidth demand so that lower-utilisation configurations of the host LLM pipeline can still sustain high decode throughput.

Domain-Specific Accelerators. Titanus [6] proposes a cascade pruning-quantization (CPQ) pipeline in computing-in-memory chiplets, reporting 159.9× energy efficiency vs. A100 for long-context inference. Kelle [7] demonstrates 3.9× speedup and 4.5× energy improvement by co-designing KV eviction (AERP) with embedded eDRAM at 84.8 pJ/B. Both target commercial silicon in GPU/DRAM serving stacks. LASSO targets an open-source ASIC process (SKY130) and a CPU-native edge inference stack, providing a complete simulation-to-tapeout methodology unavailable in either prior work.

Integer-Domain Softmax. IntAttention [32] implements a fully integer attention pipeline on Armv8 CPUs using Index-Softmax (32-entry LUT) to avoid floating-point exponentials. BAPS [33] uses 8-bit floating-point block-aware precision rescaling for softmax, halving bandwidth. These approaches are relevant to future LACU extensions beyond the v1 scope.

IX. Conclusion

We presented LASSO, a KV cache compression coprocessor for CPU-native LLM inference, with a simulation-driven pre-RTL methodology that discovered and fixed three critical datapath issues before any hardware was committed. The core contributions — UINT16 scale registers, INT64/DSP48E2 accumulator specification, and adaptive TIU threshold calibration — directly reflect findings from a 158-case parametric sweep of the Python golden model. All six original failures were resolved, reducing the sweep from 5 FAIL+1 OVERFLOW to 0 FAIL.

Performance evaluation via the golden model and analytical RTL model quantifies a 2.09× KV cache compression ratio (7.7 vs. 16 bits/element), 52.1% bandwidth reduction, and 2.09–4.08× end-to-end decode speedup on DDR4 platforms. The LACU compute roof at 300 MHz (19.2 GFLOP/s, 32 DSP48E2) places LASSO firmly in the compute-bound regime, confirming the DSP array — not DRAM bandwidth — as the binding resource. FPGA prototyping on Xilinx Zynq UltraScale+ ZCU102/ZCU104 is planned as the Nov 2026 pre-silicon validation step. The repository, golden model, and all 158 tests are open-source at `github.com/LonghornSilicon/lasso`.

References

[1] P. verify arXiv:2306.14048, "H_2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, [PLACEHOLDER] arXiv:2306.14048. Fetch BibTeX programmatically.

[2] P. verify arXiv:2404.14469, "SnapKV: LLM knows what you are looking for before generation," in *arXiv preprint*, 2024, [PLACEHOLDER] arXiv:2404.14469. Fetch BibTeX programmatically.

[3] G. Xiao, Y. Tang, J. Zuo, J. Guo, S. Yang, H. Tang, Y. Fu, and S. Han, "Efficient streaming language models with attention sinks," in *International Conference on Learning Representations (ICLR)*, 2024, [PLACEHOLDER] arXiv:2309.17453. Verify author list before submitting.

[4] J. Zhang, J. Wei, P. Zhang, J. Zhu, and J. Chen, "SageAttention: Accurate 8-bit attention for plug-and-play inference acceleration," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.02367 [VERIFIED].

[5] P. verify arXiv:2409.16997, "INT-FlashAttention: Enabling flash attention for INT8 quantization," *arXiv preprint arXiv:2409.16997*, 2024, [PLACEHOLDER] arXiv:2409.16997. Fetch BibTeX programmatically.

[6] P. verify title: Titanus KV Cache GLSVLSI 2025, "Titanus: Enabling KV cache pruning and quantization on-the-fly for LLM acceleration," in *ACM Great Lakes Symposium on VLSI (GLSVLSI)*, 2025, [PLACEHOLDER] Search Semantic Scholar for "Titanus KV Cache GLSVLSI".

[7] T. Xia and S. Q. Zhang, "Kelle: Co-design KV caching and eDRAM for efficient LLM serving in edge computing," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2025, [VERIFIED].

[8] P. verify arXiv:2503.18599, "Oaken: Fast and efficient LLM serving with online-offline hybrid KV cache quantization," in *Proceedings of the 52nd International Symposium on Computer Architecture (ISCA)*, 2025, [PLACEHOLDER] arXiv:2503.18599 ISCA 2025. CRITICAL: closest hardware competitor. Fetch BibTeX programmatically.

[9] P. verify arXiv:2604.04722, "Don't waste bits! Adaptive KV cache quantization," *arXiv preprint arXiv:2604.04722*, 2026, [PLACEHOLDER] CVPR 2026. Fetch via: curl -LH "Accept: application/x-bibtex" https://doi.org/10.48550/arXiv.2604.04722.

[10] P. verify arXiv:2504.19874, "TurboQuant: Walsh-Hadamard transform and Lloyd-Max codebook for KV cache quantization," in *International Conference on Learning Representations (ICLR)*, 2026, [PLACEHOLDER] arXiv:2504.19874. Fetch BibTeX programmatically.

[11] P. verify arXiv:2407.08608, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," *arXiv preprint arXiv:2407.08608*, 2024, [PLACEHOLDER] NeurIPS 2024. arXiv:2407.08608.

[12] P. verify arXiv:2211.10438, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning (ICML)*, 2023, [PLACEHOLDER] arXiv:2211.10438. Fetch BibTeX programmatically.

[13] NVIDIA Corporation, *NVIDIA TensorRT: Working with Quantized Types*, 2024, [PLACEHOLDER] https://docs.nvidia.com/deeplearning/tensorrt/latest/ inference-library/work-quantized-types.html. Verify URL before submitting.

[14] P. verify arXiv:2504.18929, "Revisiting transformers through the lens of low entropy and dynamic sparsity," *arXiv preprint arXiv:2504.18929*, 2025, [PLACEHOLDER] arXiv:2504.18929. Fetch BibTeX programmatically.

[15] P. verify arXiv:2504.17768, *The Sparse Frontier: Sparse Attention Trade-offs in Transformer LLMs*, 2025, [PLACEHOLDER] arXiv:2504.17768. Fetch BibTeX programmatically.

[16] A. Karpathy, "autoresearch," https://github.com/karpathy/autoresearch, 2026, [PLACEHOLDER] GitHub repository. Verify URL before submitting.

[17] P. verify arXiv:2305.17118, "Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, [PLACEHOLDER] arXiv:2305.17118. Fetch BibTeX programmatically.

[18] P. verify arXiv:2407.11550, "Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for attention heads," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, [PLACEHOLDER] arXiv:2407.11550. Fetch BibTeX programmatically.

[19] P. verify arXiv:2402.02750, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," in *International Conference on Machine Learning (ICML)*, 2024, [PLACEHOLDER] arXiv:2402.02750. Fetch via: curl -LH "Accept: application/x-bibtex" https://doi.org/10.48550/arXiv.2402.02750.

[20] P. verify arXiv:2401.18079, "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," in *Advances in Neural*

Information Processing Systems (NeurIPS), 2024, [PLACEHOLDER] arXiv:2401.18079. Fetch BibTeX programmatically.

[21] P. verify GEAR KV cache arXiv, "GEAR: An efficient KV cache compression recipe for near-lossless generative inference of LLM," *arXiv preprint arXiv:2403.05527*, 2024, [PLACEHOLDER] arXiv:2403.05527. Fetch BibTeX programmatically.

[22] P. verify arXiv:2402.04396, "QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks," in *International Conference on Learning Representations (ICLR)*, 2024, [PLACEHOLDER] arXiv:2402.04396. Fetch BibTeX programmatically.

[23] P. verify arXiv:2404.00456, "QuaRot: Outlier-free 4-bit inference in rotated LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, [PLACEHOLDER] arXiv:2404.00456. Fetch BibTeX programmatically.

[24] P. verify arXiv:2405.16406, "SpinQuant: LLM quantization with learned rotations," in *International Conference on Learning Representations (ICLR)*, 2025, [PLACEHOLDER] arXiv:2405.16406. Fetch BibTeX programmatically.

[25] P. verify arXiv:2512.00956, "WUSH: Near-optimal adaptive transforms for LLM quantization," *arXiv preprint arXiv:2512.00956*, 2025, [PLACEHOLDER] Fetch BibTeX programmatically.

[26] P. verify arXiv:2505.14201, "FLASH-D: FlashAttention with hidden softmax division," *arXiv preprint arXiv:2505.14201*, 2025, [PLACEHOLDER] Fetch BibTeX programmatically.

[27] P. verify arXiv:2511.00295, "H-FA: A hybrid floating-point and logarithmic approach to hardware accelerated FlashAttention," *arXiv preprint arXiv:2511.00295*, 2025, [PLACEHOLDER] Fetch BibTeX programmatically.

[28] P. verify arXiv:2505.14314, "Low-cost FlashAttention with fused exponential and multiplication hardware operators," *arXiv preprint arXiv:2505.14314*, 2025, [PLACEHOLDER] Fetch BibTeX programmatically.

[29] P. verify arXiv:2309.06180, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, 2023, [PLACEHOLDER] arXiv:2309.06180. Fetch BibTeX programmatically.

[30] P. verify arXiv:2409.11424, "LlamaF: An efficient llama2 architecture accelerator on embedded FPGAs," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, 2024, [PLACEHOLDER] arXiv:2409.11424. Fetch BibTeX programmatically.

[31] P. verify arXiv:2507.03308, "Hummingbird: A smaller and faster large language model accelerator on embedded FPGA," *arXiv preprint arXiv:2507.03308*, 2025, [PLACEHOLDER] arXiv:2507.03308. Cite if published before submission. Fetch BibTeX programmatically.

[32] P. verify arXiv:2511.21513, "IntAttention: A fully integer attention pipeline for efficient edge inference," *arXiv preprint arXiv:2511.21513*, 2025, [PLACEHOLDER] arXiv:2511.21513. Fetch BibTeX programmatically.

[33] P. verify arXiv:2602.02071, "BAPS: A fine-grained low-precision scheme for softmax in attention via block-aware precision rescaling," *arXiv preprint arXiv:2602.02071*, 2026, [PLACEHOLDER] arXiv:2602.02071. Fetch BibTeX programmatically.